

1. Foundation Pages

The load-bearing walls.

a) Blog Index Page

`/blog`

This is the master directory of all articles.

It needs:

- Pagination
- Filtering by category
- Sorting (newest/popular)
- A grid/list toggle (optional)
- ArticleCard components

This page communicates your entire content structure.
Once this is solid, everything else hangs off it.

b) Article Page (the full post)

`/blog/[slug]`

This is the crown jewel.

It forces you to finalize your **typography system**, long-form layout, images, footnotes, code blocks, and embedded media.

It needs:

- ArticleHeader
- ArticleBody (rich typography tokens)
- AuthorBadge
- CategoryPill
- Next/Previous navigation
- Related articles section
- Comments (optional earlier, can add later)

The article page is where your design language expresses itself most clearly.
It's also where 80% of SEO happens.

c) Category Page

`/blog/category/[category]`

This is basically “Blog Index with filters applied by default”.
You need this for proper taxonomy, breadcrumbs, and future scaling.

It needs:

- Hero section for the category

- ArticleGrid
- Pagination
- SEO metadata

d) Author Page

`/blog/author/[username]`

Your system is designed to support multiple authors and designers, so their hub pages are essential.

Author pages should include:

- Profile info (photo/name/bio/social)
- Articles by that author
- Optional: cross-section content (their videos or products in the future)

Done right, this gives your site a magazine feel instead of “just a blog”.

Build these 4 first.

Nothing else matters until these are stable.

2. Secondary Pages (after the foundation is ready)

Search Page

`/search?q=...`

Unified search for blog, video, and ecommerce later.

Build it early enough to influence component sizes and typography.

Tag Page (optional if categories are enough)

`/blog/tag/[tag]`

Useful if you want a lighter categorization layer.

Tags help SEO, but only if not abused.

Archive Page

`/blog/archive/[year]/[month]`

Purely SEO + long-term UX.
Nice to have once content grows.

3. Admin-facing pages (later)

These only matter once your CMS or internal tools come into play:

- New article form
- Article editing
- Media manager
- Writer dashboard
- Draft workflow

Your .NET backend will feed these, but we won't waste frontend cycles here yet.

How to proceed step-by-step

Here's the recommended construction sequence—clean, scalable, predictable:

Step 1: Define the content model

Even before UI—decide what an article *is*:

```
id
title
slug
excerpt
coverImage
authorId
category
tags
publishedAt
readingTime
body (markdown or structured blocks)
This ensures your frontend doesn't have to change later.
```

Step 2: Build the ArticleCard

Because it will be used everywhere—index, category, author pages.

Step 3: Build the Blog Index Page

Start with dummy data.
Wire the layout, pagination, filters.

Step 4: Build the Article Page

Force your typography to settle.
Lock down spacing, headings, blockquotes, images, code blocks.

(This step usually creates 80% of your design system adjustments.)

Step 5: Build Category & Author Pages using the same grid

Reuse components, refine as needed.

Step 6: Integrate your backend

Once the pages render correctly with mock data, plug in the .NET API.

Step 7: Add Search + SEO Enhancements

This is where you put the polished capstone on the whole structure.